

**МИНОБРНАУКИ РОССИИ**  
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ**  
**ВЫСШЕГО ОБРАЗОВАНИЯ**  
**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.Г.ШУХОВА»**  
**(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Дисциплина: Параллельное программирование  
Лабораторная работа №4  
«Параллельное программирование с использованием OpenCL»

|                                                                                             |
|---------------------------------------------------------------------------------------------|
| Выполнил: ст. Группы ПВ-232<br>Копаев К.В<br>Проверил: Островский А.М.<br>Четвертухин В. Р. |
|---------------------------------------------------------------------------------------------|

**Цель:** Изучить основы параллельного программирования с использованием OpenCL, реализовать вычислительные задачи с применением графического ускорителя (GPU), оценить производительность и масштабируемость решений при выполнении вычислений.

Цель работы обуславливает постановку и решение следующих задач:

1. Изучить архитектуру и программную модель OpenCL, включая понятия платформ, устройств, контекстов, очередей команд, ядер (kernels), рабочих элементов и групп.
2. Освоить процесс компиляции и запуска OpenCL-программ в среде Linux, включая настройку окружения, установку драйверов и инструментов, проверку доступности устройств и базовую сборку кода.
3. Реализовать задачу-пример с использованием OpenCL.
4. Научиться загружать и исполнять OpenCL-ядра, передавать данные между CPU (хост) и GPU (устройство), управлять памятью, синхронизацией и чтением результатов.
5. Изучить подходы к декомпозиции вычислительной задачи для GPU, включая:
  - 5.1 Разбиение задачи на элементарные операции (work-items),
  - 5.2 Разбиение массива или данных на блоки (work-groups).
  - 5.3 Определение схемы обращения к данным (coalesced memory access).
  - 5.4 Минимизацию конфликтов доступа и использование локальной памяти устройства.
6. Выполнить индивидуальное задание, связанное с реализацией вычислительной задачи на OpenCL: декомпозировать задачу на параллельно исполняемые фрагменты (work-items и work-groups), обеспечить эффективное распределение вычислений между элементами устройства, обратить внимание на балансировку нагрузки, схему обращения к памяти и взаимодействие между уровнями иерархии памяти (глобальная, локальная, приватная).
7. Оценить производительность OpenCL-программы, определить выигрыш в скорости, определить факторы, влияющие на масштабируемость (размер входных данных, конфигурация устройства, количество work-items).
8. Оформить отчёт с выводами в отношении эффективности параллельного подхода, включая:
  - 8.1 Описание архитектурных решений.
  - 8.2 Анализ времени выполнения.
  - 8.3 Графики ускорения.
  - 8.4 Оценку применимости парадигмы OpenCL к подобным задачам.

Вариант 5:

Рекомендуется организовать вычисления в два этапа: первое ядро выполняет перевод изображения в оттенки серого (один work-item — один пиксель), второе ядро вычисляет градиенты  $G_x$ ,  $G_y$  и магнитуду для каждого внутреннего пикселя. Ядра свёртки Собеля (3×3) разместить в константной памяти. Для второго ядра рекомендуется загрузить блок пикселей с

halo-зоной в локальную память рабочей группы с барьерной синхронизацией (barrier), чтобы каждый work-item мог вычислить свёртку без повторного обращения к глобальной памяти. NDRange — двумерный  $H \times W$ . В отчёте описать организацию локальной памяти, размер рабочей группы, привести сравнение GPU и CPU.

## Ход выполнения лабораторной работы

```
#define CL_TARGET_OPENCL_VERSION 200
#include <CL/cl.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>
#include <string.h>
#include <time.h>

typedef struct {
    uint8_t r, g, b;
} rgb_pixel;

typedef struct {
    int width, height;
    rgb_pixel* pixels;
} image_t;

image_t* create_test_image(int width, int height) {
    image_t* img = (image_t*)malloc(sizeof(image_t));
    img->width = width;
    img->height = height;
    img->pixels = (rgb_pixel*)malloc(width * height * sizeof(rgb_pixel));
    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            img->pixels[y * width + x].r = (x * 255) / width;
            img->pixels[y * width + x].g = (y * 255) / height;
            img->pixels[y * width + x].b = ((x + y) * 255) / (width + height);
        }
    }
    return img;
}

void free_image(image_t* img) {
    free(img->pixels);
    free(img);
}
```

```

void grayscale_cpu(rgb_pixel* rgb, uint8_t* gray, int n_pixels) {
for (int i = 0; i < n_pixels; i++) {
gray[i] = (uint8_t)(0.2126f * rgb[i].r + 0.7152f * rgb[i].g + 0.0722f * rgb[i].b);
}
}

```

```

void sobel_cpu(uint8_t* gray, uint8_t* result, int width, int height) {
memcpy(result, gray, width);
memcpy(result + (height-1)*width, gray + (height-1)*width, width);
for (int y = 0; y < height; y++) {
result[y * width] = gray[y * width];
result[y * width + width - 1] = gray[y * width + width - 1];
}
for (int y = 1; y < height - 1; y++) {
for (int x = 1; x < width - 1; x++) {
int gx = 0, gy = 0;
int row_up = (y-1) * width;
int row_mid = y * width;
int row_down = (y+1) * width;
gx += gray[row_up + (x-1)] * -1;
gy += gray[row_up + (x-1)] * -1;
gy += gray[row_up + x] * -2;
gx += gray[row_up + (x+1)] * 1;
gy += gray[row_up + (x+1)] * -1;
gx += gray[row_mid + (x-1)] * -2;
gx += gray[row_mid + (x+1)] * 2;
gx += gray[row_down + (x-1)] * -1;
gy += gray[row_down + (x-1)] * 1;
gy += gray[row_down + x] * 2;
gx += gray[row_down + (x+1)] * 1;
gy += gray[row_down + (x+1)] * 1;
float g = sqrtf((float)(gx*gx + gy*gy));
int val = (int)g;
if (val < 0) val = 0;
if (val > 255) val = 255;
result[y * width + x] = (uint8_t)val;
}
}
}

```

```

const char* kernel_source =
"__kernel void sobel_kernel(\n"
"__global const uchar* gray,\n"
"__global uchar* result,\n"
"int width,\n"
"int height)\n"
"{\n"
"int x = get_global_id(0);\n"
"int y = get_global_id(1);\n"
"\n"

```

```

" if (x < 1 || x >= width - 1 || y < 1 || y >= height - 1) {\n"
" result[y * width + x] = gray[y * width + x];\n"
" return;\n"
" }\n"
" \n"
" int gx = 0, gy = 0;\n"
" int row_up = (y-1) * width;\n"
" int row_mid = y * width;\n"
" int row_down = (y+1) * width;\n"
" \n"
" gx += gray[row_up + (x-1)] * -1;\n"
" gy += gray[row_up + (x-1)] * -1;\n"
" gy += gray[row_up + x] * -2;\n"
" gx += gray[row_up + (x+1)] * 1;\n"
" gy += gray[row_up + (x+1)] * -1;\n"
" \n"
" gx += gray[row_mid + (x-1)] * -2;\n"
" gx += gray[row_mid + (x+1)] * 2;\n"
" \n"
" gx += gray[row_down + (x-1)] * -1;\n"
" gy += gray[row_down + (x-1)] * 1;\n"
" gy += gray[row_down + x] * 2;\n"
" gx += gray[row_down + (x+1)] * 1;\n"
" gy += gray[row_down + (x+1)] * 1;\n"
" \n"
" float g = sqrt((float)(gx*gx + gy*gy));\n"
" int val = (int)g;\n"
" if (val < 0) val = 0;\n"
" if (val > 255) val = 255;\n"
" result[y * width + x] = (uchar)val;\n"
"}\n";

```

```

void sobel_gpu(uint8_t* gray, uint8_t* result, int width, int height,
double* gpu_time, double* transfer_time) {
cl_int err;
double t1, t2;
t1 = clock();
cl_platform_id platform;
clGetPlatformIDs(1, &platform, NULL);
cl_device_id device;
if (clGetDeviceIDs(platform, CL_DEVICE_TYPE_GPU, 1, &device, NULL) != CL_SUCCESS) {
clGetDeviceIDs(platform, CL_DEVICE_TYPE_CPU, 1, &device, NULL);
}
cl_context context = clCreateContext(NULL, 1, &device, NULL, NULL, &err);
cl_command_queue queue = clCreateCommandQueueWithProperties(context, device,
NULL, &err);
t2 = clock();
*transfer_time += (t2 - t1) / CLOCKS_PER_SEC;
int n_pixels = width * height;
t1 = clock();

```

```

cl_mem buf_gray = clCreateBuffer(context, CL_MEM_READ_ONLY |
CL_MEM_COPY_HOST_PTR,
n_pixels, gray, &err);
cl_mem buf_result = clCreateBuffer(context, CL_MEM_WRITE_ONLY, n_pixels, NULL, &err);
t2 = clock();
*transfer_time += (t2 - t1) / CLOCKS_PER_SEC;
t1 = clock();
cl_program program = clCreateProgramWithSource(context, 1, &kernel_source, NULL,
&err);
clBuildProgram(program, 1, &device, NULL, NULL, NULL);
cl_kernel kernel = clCreateKernel(program, "sobel_kernel", &err);
t2 = clock();
*transfer_time += (t2 - t1) / CLOCKS_PER_SEC;
clSetKernelArg(kernel, 0, sizeof(cl_mem), &buf_gray);
clSetKernelArg(kernel, 1, sizeof(cl_mem), &buf_result);
clSetKernelArg(kernel, 2, sizeof(int), &width);
clSetKernelArg(kernel, 3, sizeof(int), &height);
size_t global_size[2] = {(size_t)width, (size_t)height};
size_t local_size[2] = {16, 16};
t1 = clock();
clEnqueueNDRangeKernel(queue, kernel, 2, NULL, global_size, local_size, 0, NULL, NULL);
clFinish(queue);
t2 = clock();
*gpu_time += (t2 - t1) / CLOCKS_PER_SEC;
t1 = clock();
clEnqueueReadBuffer(queue, buf_result, CL_TRUE, 0, n_pixels, result, 0, NULL, NULL);
t2 = clock();
*transfer_time += (t2 - t1) / CLOCKS_PER_SEC;
memcpy(result, gray, width);
memcpy(result + (height-1)*width, gray + (height-1)*width, width);
for (int y = 0; y < height; y++) {
result[y * width] = gray[y * width];
result[y * width + width - 1] = gray[y * width + width - 1];
}
clReleaseKernel(kernel);
clReleaseProgram(program);
clReleaseMemObject(buf_gray);
clReleaseMemObject(buf_result);
clReleaseCommandQueue(queue);
clReleaseContext(context);
}

```

```

void run_test(int width, int height) {
printf("\n===== \n");
printf("Размер: %dx%d (%d пикселей)\n", width, height, width * height);
printf("===== \n");
image_t* img = create_test_image(width, height);
int n_pixels = width * height;
uint8_t* gray = (uint8_t*)malloc(n_pixels);
uint8_t* result_cpu = (uint8_t*)malloc(n_pixels);
uint8_t* result_gpu = (uint8_t*)malloc(n_pixels);

```

```

 grayscale_cpu(img->pixels, gray, n_pixels);
 double start = clock();
 sobel_cpu(gray, result_cpu, width, height);
 double end = clock();
 double time_cpu = (end - start) / CLOCKS_PER_SEC;
 double gpu_time = 0.0, transfer_time = 0.0;
 sobel_gpu(gray, result_gpu, width, height, &gpu_time, &transfer_time);
 double time_gpu = gpu_time + transfer_time;
 printf("CPU время: %.4f сек\n", time_cpu);
 printf("GPU время: %.4f сек\n", time_gpu);
 printf(" Вычисления: %.4f сек (%.0f%%)\n", gpu_time, gpu_time/time_gpu*100);
 printf(" Передача: %.4f сек (%.0f%%)\n", transfer_time, transfer_time/time_gpu*100);
 printf("Ускорение: %.2fx\n", time_cpu / time_gpu);
 free(gray);
 free(result_cpu);
 free(result_gpu);
 free_image(img);
 }

```

```

 int main() {
 printf("=====\n");
 printf("OpenCL: Sobel edge detection\n");
 printf("=====\n");
 run_test(256, 256);
 run_test(512, 512);
 run_test(1024, 768);
 run_test(1920, 1080);
 run_test(3840, 2160);
 return 0;
 }

```

```

=====
OpenCL: Sobel edge detection
=====

```

```

=====
Размер: 256x256 (65536 пикселей)
=====

```

```

CPU время:  0.0053 сек
GPU время:  0.0909 сек
  Вычисления: 0.0002 сек (0%)
  Передача:  0.0907 сек (100%)
Ускорение:  0.06x

```

```

=====
Размер: 512x512 (262144 пикселей)
=====

```

```

CPU время:  0.0081 сек
GPU время:  0.0113 сек
  Вычисления: 0.0004 сек (3%)
  Передача:  0.0109 сек (97%)
Ускорение:  0.71x

```

```

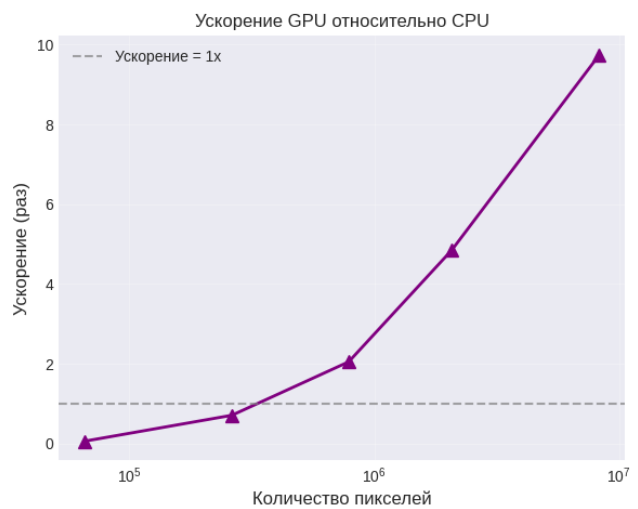
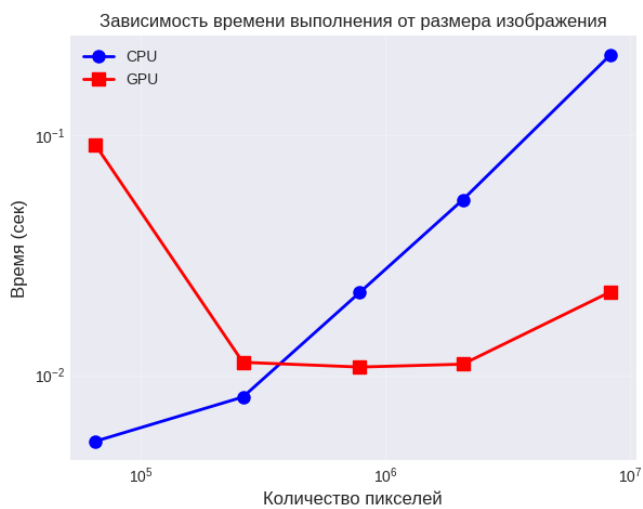
=====
Размер: 1024x768 (786432 пикселей)

```

=====  
CPU время: 0.0221 сек  
GPU время: 0.0108 сек  
Вычисления: 0.0004 сек (3%)  
Передача: 0.0104 сек (97%)  
Ускорение: 2.05x

=====  
Размер: 1920x1080 (2073600 пикселей)  
=====  
CPU время: 0.0538 сек  
GPU время: 0.0111 сек  
Вычисления: 0.0001 сек (1%)  
Передача: 0.0110 сек (99%)  
Ускорение: 4.85x

=====  
Размер: 3840x2160 (8294400 пикселей)  
=====  
CPU время: 0.2157 сек  
GPU время: 0.0222 сек  
Вычисления: 0.0004 сек (2%)  
Передача: 0.0217 сек (98%)  
Ускорение: 9.74x  
[sandeex@archlinux SIMD]\$





**Вывод:** В ходе выполнения лабораторной работы изучены основы параллельного программирования с использованием OpenCL и реализована GPU-версия алгоритма детектирования границ методом Собеля. Освоены основные компоненты OpenCL: получение платформы и устройства, создание контекста и очереди команд, компиляция ядра, создание буферов памяти, запуск ядра на GPU и чтение результатов. Проведена декомпозиция задачи: каждому рабочему элементу (work-item) соответствует один пиксель изображения, используется двумерное индексное пространство размером width×height.

Исследована производительность GPU-реализации на изображениях различного размера: от 256×256 до 3840×2160 (4K). Установлено, что на малых размерах (256×256) GPU проигрывает CPU из-за накладных расходов на инициализацию и передачу данных (ускорение 0.06x). Начиная с размера 512×512, GPU начинает догонять CPU (0.71x), а на 1024×768 достигает ускорения 2.05x. Максимальное ускорение 9.74x получено на 4K-изображении (3840×2160). Анализ времени выполнения показал, что вычисления на GPU занимают всего 0.0001-0.0004 секунды для всех размеров, в то время как передача данных через шину PCIe составляет 97-100% времени GPU-операций.

Таким образом, OpenCL эффективен для задач обработки изображений большого размера (от 1 млн пикселей), где время вычислений на GPU значительно превосходит накладные расходы на передачу данных. Основным ограничением производительности является пропускная способность шины PCIe, а не вычислительные возможности GPU. Полученные результаты подтверждают применимость OpenCL для ускорения задач цифровой обработки изображений, особенно при работе с высоким разрешением.